04. 컴파일 과정 (Compilation Process)

개요

컴파일 과정은 Monkey 언어의 AST(Abstract Syntax Tree)를 바이트코드(Bytecode)로 변환하는 단계입니다. 이 과정에서 `Compiler` 구조체가 핵심 역할을 하며, AST 노드들을 순회하면서 각각에 해당하는 바이트코드 명령어를 생성합니다. 컴파일러는 인터프리터와 달리 실행 전에 모든 코드를 바이트코드로 변환하여 가상 머신(VM)에서 실행할 수 있도록 준비합니다.

1. Compiler 구조체와 컴파일 개념

1.1 Compiler 구조체의 구성

`Compiler` 구조체는 컴파일 과정에서 필요한 모든 정보를 관리합니다:

```
type Compiler struct {
instructions code.Instructions // 생성된 바이트코드 명령어들
constants []object.Object // 상수 풀 (정수, 문자열 등)
lastInstruction EmittedInstruction // 마지막으로 생성된 명령어
previousInstruction EmittedInstruction // 이전에 생성된 명령어
symbolTable *SymbolTable // 심볼 테이블 (변수 관리)
scopes []CompilationScope // 스코프 스택
scopeIndex  int  // 현재 스코프 인덱스
}
```

각 필드의 역할:

- **instructions**: 컴파일된 바이트코드 명령어들이 저장되는 바이트 슬라이스
- **constants**: 컴파일 시점에 결정되는 상수들 (정수 리터럴, 문자열 등)을 저장
- **lastInstruction/previousInstruction**: 최적화를 위해 최근 생성된 명령어들을 추적
- **symbolTable**: 변수와 함수의 스코프 및 위치 정보를 관리
- **scopes**: 함수 스코프별로 별도의 컴파일 컨텍스트를 유지
- **scopeIndex**: 현재 활성화된 스코프의 인덱스

1.2 컴파일러 초기화

새로운 컴파일러 인스턴스는 `New()` 함수로 생성됩니다:

```
func New() *Compiler {
mainScope := CompilationScope{
instructions: code.Instructions{},
lastInstruction: EmittedInstruction{},
previousInstruction: EmittedInstruction{},
}

symbolTable := NewSymbolTable()

// 내장 함수들을 심볼 테이블에 등록
for i, v := range object.Builtins {
symbolTable.DefineBuiltin(i, v.Name)
}
```

```
return &Compiler{
constants: []object.Object{},
symbolTable: symbolTable,
scopes: []CompilationScope{mainScope},
scopeIndex: 0,
}
}
```

초기화 과정:

1. **메인 스코프 생성**: 전역 스코프에 해당하는 `CompilationScope` 생성
2. **심볼 테이블 초기화**: 변수와 함수 이름을 관리할 심볼 테이블 생성
3. **내장 함수 등록**: `len`, `first`, `last` 등의 내장 함수들을 심볼 테이블에 미리 등록
4. **빈 상수 풀**: 컴파일 과정에서 발견되는 상수들을 저장할 슬라이스 초기화

1.3 AST에서 바이트코드로의 변환 개념

컴파일 과정은 다음과 같은 단계로 이루어집니다:

1.3.1 재귀적 AST 순회

`Compile()` 함수는 AST 노드를 재귀적으로 순회하며 각 노드 타입에 맞는 바이트코드를 생성합니다:

```
func (c *Compiler) Compile(node ast.Node) error {
switch node := node.(type) {
case *ast.Program:
```

```go
// 프로그램의 모든 문장을 순회하며 컴파일
for _, s := range node.Statements {
err := c.Compile(s)
if err != nil {
return err
}
}
case *ast.ExpressionStatement:
// 표현식을 컴파일하고 결과를 스택에서 제거
err := c.Compile(node.Expression)
if err != nil {
return err
}
c.emit(code.OpPop)
// ... 다른 노드 타입들
}
return nil
}
```

1.3.2 명령어 생성과 상수 관리

컴파일러는 두 가지 주요 작업을 수행합니다:

명령어 생성: `emit()` 함수를 통해 바이트코드 명령어를 생성

```go
func (c *Compiler) emit(op code.Opcode, operands ...int) int {
ins := code.Make(op, operands...) // 바이트코드 명령어 생성
pos := c.addInstruction(ins) // 명령어를 instructions에 추가
c.setLastInstruction(op, pos) // 최근 명령어 정보 업데이트
return  pos
}
```

상수 관리: 리터럴 값들을 상수 풀에 저장하고 인덱스로 참조

```go
func (c *Compiler) addConstant(obj object.Object) int {
c.constants = append(c.constants, obj) // 상수 풀에 추가
return  len(c.constants) - 1  // 인덱스 반환
}
```

1.3.3 컴파일 결과 생성

컴파일이 완료되면 `Bytecode` 구조체로 결과를 반환합니다:

```go
type Bytecode struct {
Instructions code.Instructions // 바이트코드 명령어 시퀀스
Constants []object.Object // 상수 풀
}
```

```go
func (c *Compiler) Bytecode() *Bytecode {
return &Bytecode{
```

```
Instructions: c.currentInstructions(),

Constants: c.constants,

}

}
```

이 `Bytecode` 구조체는 가상 머신(VM)에서 실행될 수 있는 완전한 형태의 컴파일된 프로그램을 나타냅니다.

1.4 인터프리터와의 차이점

컴파일러 방식의 주요 특징:

1. **사전 변환**: 실행 전에 모든 코드를 바이트코드로 변환
2. **최적화 가능**: 컴파일 시점에 코드 최적화 수행 가능
3. **빠른 실행**: VM에서 바이트코드를 직접 실행하여 속도 향상
4. **메모리 효율**: 바이트코드는 AST보다 메모리 사용량이 적음
5. **포터빌리티**: 바이트코드는 플랫폼 독립적으로 실행 가능

2. 바이트코드와 Opcode 시스템

2.1 Opcode 상수와 명령어 체계

Monkey 컴파일러는 다양한 연산과 제어 흐름을 위한 바이트코드 명령어들을 정의합니다:

```
type Opcode byte


const (

OpConstant Opcode = iota // 상수 로드
```

```
OpPop  // 스택에서 값 제거

OpAdd  // 덧셈 연산

OpSub  // 뺄셈 연산

OpMul  // 곱셈 연산

OpDiv  // 나눗셈 연산

OpTrue  // true 값 푸시

OpFalse  // false 값 푸시

OpEqual  // 동등 비교 (==)

OpNotEqual  // 부등 비교 (!=)

OpGreaterThan  // 크기 비교 (>)

OpMinus  // 단항 마이너스 (-)

OpBang  // 논리 부정 (!)

OpJumpNotTruthy  // 조건부 점프 (거짓일 때)

OpJump  // 무조건 점프

OpNull  // null 값 푸시

OpGetGlobal  // 전역 변수 읽기

OpSetGlobal  // 전역 변수 설정

OpArray  // 배열 생성

OpHash  // 해시 생성

OpIndex  // 인덱스 접근

OpCall  // 함수 호출

OpReturnValue  // 값과 함께 반환

OpReturn  // 값 없이 반환
```

```
OpGetLocal  // 지역 변수 읽기
OpSetLocal  // 지역 변수 설정
OpGetBuiltin  // 내장 함수 읽기
OpClosure  // 클로저 생성
OpGetFree  // 자유 변수 읽기
)
```

2.2 명령어 정의와 피연산자 구조

각 명령어는 `Definition` 구조체로 정의되며, 피연산자의 크기 정보를 포함합니다:

```
type Definition struct {
Name  string  // 명령어 이름
OperandWidths []int  // 각 피연산자의 바이트 크기
}

var  definitions = map[Opcode]*Definition{
OpConstant: {"OpConstant", []int{2}}, // 2바이트 상수 인덱스
OpPop: {"OpPop", []int{}}, // 피연산자 없음
OpAdd: {"OpAdd", []int{}}, // 피연산자 없음
OpGetGlobal: {"OpGetGlobal", []int{2}}, // 2바이트 변수 인덱스
OpSetGlobal: {"OpSetGlobal", []int{2}}, // 2바이트 변수 인덱스
OpCall: {"OpCall", []int{1}}, // 1바이트 인수 개수
OpGetLocal: {"OpGetLocal", []int{1}}, // 1바이트 지역 변수 인덱스
```

```
OpClosure: {"OpClosure", []int{2, 1}}, // 2바이트 함수 인덱스 + 1바이트 자유 변수 개수
// ... 기타 명령어들
}
```

2.2.1 피연산자 크기의 의미

- **2바이트 피연산자**: 최대 65,535개의 상수나 전역 변수를 지원
- **1바이트 피연산자**: 최대 255개의 지역 변수나 함수 인수를 지원
- **피연산자 없음**: 스택의 최상위 값들만 사용하는 연산

2.3 Instructions 타입과 바이트코드 구조

`Instructions` 는 바이트코드 명령어들의 시퀀스를 나타내는 바이트 슬라이스입니다:

```go
type Instructions []byte
```

2.3.1 바이트코드 구조 예시

간단한 표현식 `1 + 2` 의 컴파일 결과:

```
바이트코드:
[OpConstant, 0, 0, OpConstant, 0, 1, OpAdd, OpPop]

상수 풀:
[Integer{1}, Integer{2}]

실행 과정:
```

```
1. OpConstant 0 0 -> 상수 풀[0] (값: 1)을 스택에 푸시

2. OpConstant 0 1 -> 상수 풀[1] (값: 2)을 스택에 푸시

3. OpAdd -> 스택에서 두 값을 팝하여 더하고 결과를 푸시

4. OpPop -> 결과를 스택에서 제거 (표현식 문장이므로)
```

2.4 emit() 함수를 통한 명령어 생성 과정

`emit()` 함수는 바이트코드 명령어를 생성하고 컴파일러의 상태를 업데이트합니다:

```go
func (c *Compiler) emit(op code.Opcode, operands ...int) int {
ins := code.Make(op, operands...) // 바이트코드 명령어 생성
pos := c.addInstruction(ins) // 명령어를 instructions에 추가
c.setLastInstruction(op, pos) // 최근 명령어 정보 업데이트
return pos // 명령어의 위치 반환
}
```

2.4.1 Make() 함수의 바이트코드 생성

`code.Make()` 함수는 Opcode와 피연산자들을 실제 바이트 배열로 변환합니다:

```go
func Make(op Opcode, operands ...int) []byte {
def, ok := definitions[op]
if !ok {
return []byte{}
}
```

```go
// 명령어 전체 길이 계산 (Opcode 1바이트 + 피연산자들)
instructionLen := 1
for _, w := range def.OperandWidths {
instructionLen += w
}

instruction := make([]byte, instructionLen)
instruction[0] = byte(op) // 첫 번째 바이트는 Opcode

// 피연산자들을 빅엔디안 형식으로 인코딩
offset := 1
for i, o := range operands {
width := def.OperandWidths[i]
switch width {
case 2:
binary.BigEndian.PutUint16(instruction[offset:], uint16(o))
case 1:
instruction[offset] = byte(o)
}
offset += width
}
```

```
return instruction
}
```

2.4.2 명령어 추가와 위치 추적

생성된 명령어는 현재 스코프의 instructions에 추가됩니다:

```
func (c *Compiler) addInstruction(ins []byte) int {
posNewInstruction := len(c.currentInstructions())
updatedInstructions := append(c.currentInstructions(), ins...)
c.scopes[c.scopeIndex].instructions = updatedInstructions
return posNewInstruction // 새 명령어의 시작 위치 반환
}
```

2.5 바이트코드 최적화 기법

컴파일러는 생성된 바이트코드를 최적화하기 위한 여러 기법을 사용합니다:

2.5.1 불필요한 OpPop 제거

조건문에서 마지막 표현식의 OpPop을 제거하여 값을 유지합니다:

```
func (c *Compiler) removeLastPop() {
last := c.scopes[c.scopeIndex].lastInstruction
previous := c.scopes[c.scopeIndex].previousInstruction

old := c.currentInstructions()
```

```
new := old[:last.Position] // 마지막 OpPop 명령어 제거


c.scopes[c.scopeIndex].instructions = new
c.scopes[c.scopeIndex].lastInstruction = previous
}
```

2.5.2 점프 주소 패치

조건문과 반복문에서 점프 주소를 나중에 올바른 값으로 수정합니다:

```
func (c *Compiler) changeOperand(opPos int, operand int) {
op := code.Opcode(c.currentInstructions()[opPos])
newInstruction := code.Make(op, operand)
c.replaceInstruction(opPos, newInstruction) // 기존 명령어를 새 명령어로 교체
}
```

2.6 바이트코드 디스어셈블리

생성된 바이트코드는 디버깅을 위해 사람이 읽을 수 있는 형태로 출력할 수 있습니다:

```
func (ins Instructions) String() string {
var out bytes.Buffer


i := 0
for i < len(ins) {
def, err := Lookup(ins[i]) // Opcode 정의 조회
```

```
if  err != nil {
fmt.Fprintf(&out, "ERROR: %s\n", err)
continue
}

operands, read := ReadOperands(def, ins[i+1:]) // 피연산자 읽기
fmt.Fprintf(&out, "%04d %s\n", i, ins.fmtInstruction(def, operands))

i += 1 + read  // 다음 명령어로 이동
}

return  out.String()
}
```

출력 예시:

```
0000 OpConstant 0
0003 OpConstant 1
0006 OpAdd
0007 OpPop
```#
3. SymbolTable과 심볼 관리 시스템

3.1 SymbolTable 구조체와 심볼 정의

`SymbolTable`은 컴파일 과정에서 변수와 함수의 이름, 스코프, 위치 정보를 관리하는 핵심 컴

```go
type SymbolTable struct {
Outer *SymbolTable // 외부 스코프의 심볼 테이블 (스코프 체인)
store map[string]Symbol // 심볼 이름과 정보를 매핑하는 맵
numDefinitions int // 현재 스코프에서 정의된 심볼의 개수
FreeSymbols []Symbol // 클로저에서 사용하는 자유 변수들
}
```

3.1.1 Symbol 구조체

각 심볼은 이름, 스코프, 인덱스 정보를 포함합니다:

```go
type Symbol struct {
Name  string  // 심볼의 이름 (변수명, 함수명)
Scope SymbolScope // 심볼의 스코프 (Global, Local, Builtin, Free)
Index  int  // 스코프 내에서의 인덱스 위치
}
```

3.1.2 스코프 타입 정의

Monkey 언어는 네 가지 스코프 타입을 지원합니다:

```go
type SymbolScope string


const (
GlobalScope SymbolScope = "Global"  // 전역 스코프
LocalScope SymbolScope = "Local"  // 지역 스코프 (함수 내부)
BuiltinScope SymbolScope = "Builtin"  // 내장 함수 스코프
FreeScope SymbolScope = "Free"  // 자유 변수 스코프 (클로저)
)
```

3.2 심볼 테이블 초기화와 생성

3.2.1 새로운 심볼 테이블 생성

```go
func  NewSymbolTable() *SymbolTable {
s := make(map[string]Symbol)
free := []Symbol{}
return &SymbolTable{store: s, FreeSymbols: free}
}
```

3.2.2 중첩된 스코프를 위한 심볼 테이블

함수 정의 시 새로운 스코프가 생성되며, 외부 스코프에 대한 참조를 유지합니다:

```go
func  NewEnclosedSymbolTable(outer *SymbolTable) *SymbolTable {
s := NewSymbolTable()
```

```
s.Outer = outer  // 외부 스코프 연결
return  s
}
```

3.3 심볼 정의와 해결 과정

3.3.1 심볼 정의 (Define)

새로운 변수나 함수가 선언될 때 심볼을 정의합니다:

```
func (s *SymbolTable) Define(name  string) Symbol {
symbol := Symbol{Name: name, Index: s.numDefinitions}
// 스코프 결정: 외부 스코프가 없으면 전역, 있으면 지역
if  s.Outer == nil {
symbol.Scope = GlobalScope
} else {
symbol.Scope = LocalScope
}

s.store[name] = symbol
s.numDefinitions++
return  symbol
}
```

예시: `let x = 5;` 컴파일 시

1. `Define("x")` 호출
2. 전역 스코프라면 `Symbol{Name: "x", Scope: GlobalScope, Index: 0}` 생성
3. `OpSetGlobal 0` 명령어 생성

3.3.2 내장 함수 정의

내장 함수들은 컴파일러 초기화 시 미리 정의됩니다:

```
func (s *SymbolTable) DefineBuiltin(index int, name string) Symbol {
symbol := Symbol{Name: name, Index: index, Scope: BuiltinScope}
s.store[name] = symbol
return symbol
}
```

내장 함수 등록 과정:

```
// 컴파일러 초기화 시
for i, v := range object.Builtins {
symbolTable.DefineBuiltin(i, v.Name)
}
// 결과: len, first, last, rest, push 등이 BuiltinScope로 등록
```

3.3.3 심볼 해결 (Resolve)

변수나 함수 사용 시 해당 심볼의 정보를 찾습니다:

```
func (s *SymbolTable) Resolve(name string) (Symbol, bool) {
```

```
obj, ok := s.store[name]
if !ok && s.Outer != nil {
// 현재 스코프에 없으면 외부 스코프에서 검색
obj, ok = s.Outer.Resolve(name)
if !ok {
return obj, ok
}

// 전역이나 내장 함수는 그대로 반환
if obj.Scope == GlobalScope || obj.Scope == BuiltinScope {
return obj, ok
}

// 지역 변수는 자유 변수로 변환 (클로저)
free := s.defineFree(obj)
return free, true
}
return obj, ok
}
```

3.4 스코프별 심볼 관리와 처리

3.4.1 전역 스코프 (GlobalScope)

전역 변수는 프로그램 전체에서 접근 가능하며, 인덱스로 관리됩니다:

```go
// 전역 변수 설정: let globalVar = 10;
case *ast.LetStatement:
symbol := c.symbolTable.Define(node.Name.Value)
err := c.Compile(node.Value)
if err != nil {
return err
}
if symbol.Scope == GlobalScope {
c.emit(code.OpSetGlobal, symbol.Index) // OpSetGlobal 0
}

// 전역 변수 읽기: globalVar
case *ast.Identifier:
symbol, ok := c.symbolTable.Resolve(node.Value)
if !ok {
return fmt.Errorf("undefined variable %s", node.Value)
}
if symbol.Scope == GlobalScope {
c.emit(code.OpGetGlobal, symbol.Index) // OpGetGlobal 0
}
```

3.4.2 지역 스코프 (LocalScope)

함수 내부의 변수들은 지역 스코프에서 관리됩니다:

```
// 함수 컴파일 시 새로운 스코프 생성
case *ast.FunctionLiteral:
c.enterScope() // 새로운 스코프 진입
// 매개변수들을 지역 변수로 정의
for _, p := range node.Parameters {
c.symbolTable.Define(p.Value)
}
err := c.Compile(node.Body)
// ...
instructions := c.leaveScope() // 스코프 종료
```

지역 변수 접근:

```
func (c *Compiler) loadSymbol(s Symbol) {
switch s.Scope {
case LocalScope:
c.emit(code.OpGetLocal, s.Index) // OpGetLocal 0, 1, 2...
case GlobalScope:
c.emit(code.OpGetGlobal, s.Index) // OpGetGlobal 0, 1, 2...
// ...
}
}
```

3.4.3 내장 함수 스코프 (BuiltinScope)

내장 함수들은 특별한 스코프에서 관리되며, 별도의 인덱스를 가집니다:

```
// 내장 함수 호출: len(array)
case BuiltinScope:
c.emit(code.OpGetBuiltin, s.Index) // OpGetBuiltin 0 (len 함수)
```

3.4.4 자유 변수 스코프 (FreeScope) - 클로저 지원

클로저에서 외부 스코프의 변수를 참조할 때 자유 변수로 처리됩니다:

```
func (s *SymbolTable) defineFree(original Symbol) Symbol {
s.FreeSymbols = append(s.FreeSymbols, original)

symbol := Symbol{
Name: original.Name,
Index: len(s.FreeSymbols) - 1,
Scope: FreeScope,
}

s.store[original.Name] = symbol
return symbol
}
```

클로저 생성 과정:

```go
case *ast.FunctionLiteral:
// ... 함수 컴파일 ...
freeSymbols := c.symbolTable.FreeSymbols
// 자유 변수들을 스택에 로드
for _, s := range freeSymbols {
c.loadSymbol(s)
}
// 클로저 생성 명령어
c.emit(code.OpClosure, fnIndex, len(freeSymbols))
```

3.5 스코프 관리와 컴파일 컨텍스트

3.5.1 스코프 진입과 종료

함수 컴파일 시 새로운 컴파일 스코프를 생성합니다:

```go
func (c *Compiler) enterScope() {
scope := CompilationScope{
instructions: code.Instructions{},
lastInstruction: EmittedInstruction{},
previousInstruction: EmittedInstruction{},
}
c.scopes = append(c.scopes, scope)
c.scopeIndex++
```

```go
c.symbolTable = NewEnclosedSymbolTable(c.symbolTable)
}

func (c *Compiler) leaveScope() code.Instructions {
instructions := c.currentInstructions()
c.scopes = c.scopes[:len(c.scopes)-1]
c.scopeIndex--
c.symbolTable = c.symbolTable.Outer
return  instructions
}
```

3.5.2 스코프별 명령어 관리

각 스코프는 독립적인 명령어 시퀀스를 가집니다:

```go
type CompilationScope struct {
instructions code.Instructions // 현재 스코프의 명령어들
lastInstruction EmittedInstruction // 마지막 명령어
previousInstruction EmittedInstruction // 이전 명령어
}

func (c *Compiler) currentInstructions() code.Instructions {
return  c.scopes[c.scopeIndex].instructions
}
```

이러한 심볼 테이블 시스템을 통해 Monkey 컴파일러는 복잡한 스코프 규칙과 클로저를 정확하게 처리할 수 있습니다.